

Cours 27 — Le Web interactif : HTTP, formulaires, JavaScript

Semaine 27 ■ Manuel Hachette ch. 11 (§ 11.3) ■ 2 × 1 h

Objectifs

- Décrire une requête et une réponse HTTP
- Distinguer les méthodes GET et POST d'un formulaire
- Réagir à un événement avec JavaScript, côté client

1. Le dialogue HTTP

Quand le navigateur ouvre `http://www.lycee.fr/nsi/page.html`, il envoie au serveur (résolu par DNS, acheminé par IP — semaine 25) une **requête** texte :

```
GET /nsi/page.html HTTP/1.1
Host: www.lycee.fr
```

Le serveur renvoie une **réponse** : un code, des en-têtes, puis le contenu :

```
HTTP/1.1 200 OK
Content-Type: text/html
<!DOCTYPE html> ...
```

Codes à connaître : **200** (OK), **404** (introuvable), **403** (interdit — les droits, semaine 23!), **500** (erreur du serveur). Chaque image ou fichier CSS de la page déclenche sa propre requête.

2. Les formulaires : le client envoie des données

```
<form action="recherche.html" method="get">
  <label>Ville : <input type="text" name="ville"></label>
  <input type="submit" value="Chercher">
</form>
```

À l'envoi, deux **méthodes** :

- **GET** : les données partent **dans l'URL** — `recherche.html?ville=Lyon`. Visible, mémorisable dans les favoris ; pour les **consultations**.
- **POST** : les données partent dans le **corps** de la requête, pas dans l'URL ; pour ce qui **modifie** quelque chose ou transporte des données sensibles (mot de passe — jamais en GET!).

3. JavaScript : le programme dans la page

Le HTML structure, le CSS présente, **JavaScript agit** — il s'exécute **dans le navigateur** (côté client). Et son modèle est celui de la semaine 7 : les **événements**.

```
<button id="bouton">Cliquer</button>
<p id="resultat"></p>

<script>
function reagir() {
  let texte = document.getElementById("resultat");
  texte.textContent = "Bravo, premier clic !";
}
```

```

}
document.getElementById("bouton").onclick = reagir;
</script>

```

À lire avec les yeux de Python : `function = def`, `let` = une affectation, et `bouton.onclick = reagir` — la fonction **sans parenthèses**, exactement comme `turtle.onscreenclick(saute)`. Le `document` est la page vue comme un arbre (encore lui) que le script peut lire et **modifier** : c'est ainsi que les pages « réagissent » sans recharger.

4. Qui fait quoi ? Client et serveur

	côté client (navigateur)	côté serveur
langage	JavaScript	au choix (Python...)
voit	la page, vos clics	la requête, sa base de données
usage type	vérifier un champ, animer	enregistrer, chercher, calculer

Vérifier un formulaire côté client améliore le confort... mais ne protège rien : la vraie validation reste côté serveur (un client peut envoyer ce qu'il veut!).

À retenir

- HTTP : requête (méthode + chemin) / réponse (code + contenu); 200, 404, 403, 500.
- GET : paramètres dans l'URL (consulter); POST : dans le corps (modifier, données sensibles).
- JavaScript côté client : gestionnaires d'événements (`onclick = fonction`, sans parenthèses) qui modifient l'arbre de la page.
- Confort côté client, sécurité côté serveur.